Вариант №22

Цель работы: Изучить основы работы с процессами в ОС Linux, а также освоить сигналы как универсальное средство межпроцессного взаимодействия.

1. Изучить основные понятия о процессах в ОС Linux, относящиеся к их созданию и управлению.

2. Ознакомиться с базовыми системными вызовами POSIX для работы с процессами: `fork()`, `wait()`, `waitpid()`.

3. Изучить назначение и возможности сигналов в Linux, их классификацию (системные, пользовательские) и обработку.

4. Освоить установку обработчиков сигналов с помощью `signal()` и `sigaction()`, а также способы отправки сигналов с помощью `kill()` и `sigqueue()`.

5. Познакомиться с соответствующими консольными инструментами для мониторинга и управления процессами и сигналами.

   5.1 Изучение команд `ps`, `top`, `htop` — просмотр списка процессов, их состояний и ресурсов.

   5.2 Изучение команд `jobs`, `bg`, `fg` — управление фоновыми и приостановленными задачами в оболочке.

   5.3 Изучение команд `kill`, `pkill`, `killall` — отправка сигналов процессам по PID или имени.

   5.4 Изучение команды `ulimit` — просмотр и установка ресурсных ограничений (например, число процессов, размер стека).

   5.5 Изучение команды `strace` — отслеживание системных вызовов процесса, включая работу с сигналами.

6. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).

7. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Задание №6 Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/mman.h>

#define N          3
#define OPEN_SEC   2
#define CLOSE_SEC  3

static volatile sig_atomic_t *sent1, *sent2;
static volatile sig_atomic_t recv1 = 0, recv2 = 0, stop = 0;
static pid_t kid[N];
static sigset_t blk, old, empty;

static void h_usr1(int s)  { (void)s; recv1++; }
static void h_usr2(int s)  { (void)s; recv2++; }
static void h_int(int s)   { (void)s; stop = 1; }
static void h_alarm(int s) { (void)s; }
static void h_term(int s)  { (void)s; _exit(0); }

static void install(int signo, void (*h)(int), int flags)
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = h;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = flags;
    sigaction(signo, &sa, NULL);
}

static void setup_shared_memory(void)
```

```
38  {
39      size_t sz = N * sizeof(sig_atomic_t);
40
41      sent1 = mmap(NULL, sz, PROT_READ | PROT_WRITE,
42                   MAP_SHARED | MAP_ANONYMOUS, -1, 0);
43      sent2 = mmap(NULL, sz, PROT_READ | PROT_WRITE,
44                   MAP_SHARED | MAP_ANONYMOUS, -1, 0);
45
46      memset((void *)sent1, 0, sz);
47      memset((void *)sent2, 0, sz);
48  }
49
50  static void cleanup_shared_memory(void)
51  {
52      size_t sz = N * sizeof(sig_atomic_t);
53      munmap((void *)sent1, sz);
54      munmap((void *)sent2, sz);
55  }
56
57  static void setup_parent_handlers(void)
58  {
59      install(SIGUSR1, h_usr1,  SA_RESTART);
60      install(SIGUSR2, h_usr2,  SA_RESTART);
61      install(SIGINT,  h_int,   SA_RESTART);
62      install(SIGALRM, h_alarm, 0);
63  }
64
65  static void setup_child_handlers(void)
66  {
67      install(SIGINT,  SIG_IGN, 0);
68      install(SIGUSR1, SIG_IGN, 0);
69      install(SIGUSR2, SIG_IGN, 0);
70      install(SIGTERM, h_term,  0);
71  }
72
73  static void setup_signal_masks(void)
74  {
75      sigemptyset(&blk);
76      sigaddset(&blk, SIGUSR1);
77      sigaddset(&blk, SIGUSR2);
78      sigemptyset(&empty);
79  }
80
```

```c
static void child_loop(int id)
{
    srand((unsigned)(getpid() ^ (id * 2654435761u)));
    for (;;) {
        usleep((useconds_t)((100 + rand() % 301) * 1000));
        if (rand() & 1) {
            sent1[id]++;
            kill(getppid(), SIGUSR1);
        } else {
            sent2[id]++;
            kill(getppid(), SIGUSR2);
        }
    }
}

static void child_main(int id)
{
    setup_child_handlers();
    child_loop(id);
}

static void spawn_children(void)
{
    for (int i = 0; i < N; i++) {
        kid[i] = fork();
        if (kid[i] == 0)
            child_main(i);
    }
}

static void sum_sent(int *s1, int *s2)
{
    *s1 = 0; *s2 = 0;
    for (int i = 0; i < N; i++) { *s1 += sent1[i]; *s2 += sent2[i]; }
}

static void print_snapshot(const char *label)
{
    int s1, s2;
    sum_sent(&s1, &s2);
    printf("%s recv=(%d,%d) sent=(%d,%d)\n",
           label, recv1, recv2, s1, s2);
}
```

```
124
125 static void phase_open(void)
126 {
127     print_snapshot("[OPEN]   начало:");
128
129     time_t t0 = time(NULL);
130     while (!stop && time(NULL) - t0 < OPEN_SEC) {
131         alarm(1);
132         sigsuspend(&empty);
133     }
134     alarm(0);
135
136     print_snapshot("[OPEN]   конец: ");
137 }
138
139 static void phase_closed(void)
140 {
141     sigprocmask(SIG_BLOCK, &blk, &old);
142
143     printf("[CLOSED] сигналы заблокированы на %d с...\n", CLOSE_SEC);
144
145     struct timespec ts = { CLOSE_SEC, 0 };
146     while (nanosleep(&ts, &ts) == -1 && errno == EINTR)
147         ;
148
149     sigset_t pend;
150     sigpending(&pend);
151
152     printf("[CLOSED] pending: USR1=%d USR2=%d\n",
153            sigismember(&pend, SIGUSR1) != 0,
154            sigismember(&pend, SIGUSR2) != 0);
155
156     int b1 = recv1, b2 = recv2;
157     sigprocmask(SIG_SETMASK, &old, NULL);
158
159     struct timespec pause = { 0, 200 * 1000 * 1000 };
160     nanosleep(&pause, NULL);
161
162     printf("[UNBLK]  из pending доставлено: USR1=%d USR2=%d\n",
163            recv1 - b1, recv2 - b2);
164 }
165
166 static void run_cycle(int n)
```

```
167 {
168     printf("\nЦикл %d\n", n);
169     phase_open();
170     phase_closed();
171 }
172
173 static void shutdown_children(void)
174 {
175     install(SIGINT, SIG_IGN, 0);
176
177     for (int i = 0; i < N; i++)
178         kill(kid[i], SIGTERM);
179
180     for (int i = 0; i < N; i++)
181         while (waitpid(kid[i], NULL, 0) == -1 && errno == EINTR)
182             ;
183 }
184
185 static void print_final_report(void)
186 {
187     int s1, s2;
188     sum_sent(&s1, &s2);
189     printf("\nИтог: sent=(%d,%d) recv=(%d,%d) потеряно=(%d,%d)\n",
190            s1, s2, recv1, recv2, s1 - recv1, s2 - recv2);
191     printf("Обычные сигналы не встают в очередь: пока бит pending\n");
192     printf("установлен, повторные такие же сигналы сливаются с ним.\n");
193 }
194
195 int main(void)
196 {
197     setvbuf(stdout, NULL, _IONBF, 0);
198
199     setup_shared_memory();
200     setup_parent_handlers();
201     setup_signal_masks();
202     spawn_children();
203
204     printf("Диспетчер %d, детей: %d\n", (int)getpid(), N);
205
206     int cycle = 1;
207     while (!stop)
208         run_cycle(cycle++);
209
```

```
210     shutdown_children();
211     print_final_report();
212     cleanup_shared_memory();
213     return 0;
214 }
```

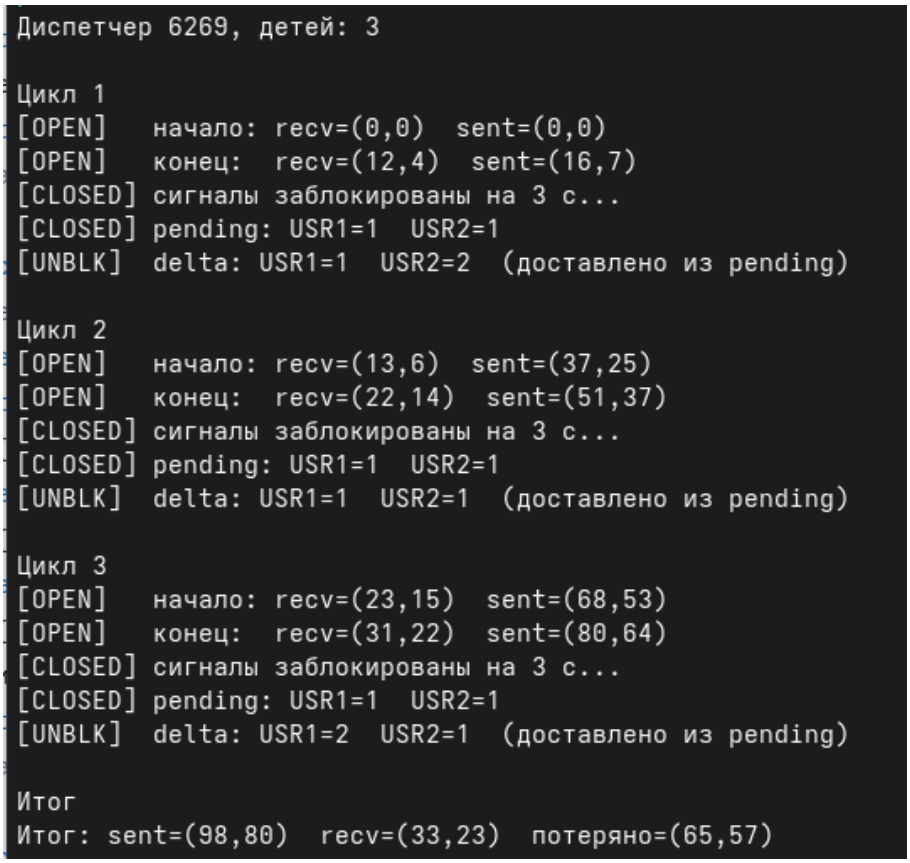


Рис. 1: Результат выполнения программы

Задание №7 Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Ниже перечислены все системные вызовы, библиотечные функции и структуры, задействованные в программе «Диспетчер с закрытым окном», с кратким описанием их назначения.

1. **Управление процессами**

   (a) `fork()` — создаёт новый процесс-потомок, копирующий адресное пространство родителя (механизм copy-on-write). Возвращает PID потомка в родителе, 0 в потомке и -1 при ошибке. Используется для порождения N исполнителей.

   (b) `_exit(status)` — немедленно завершает процесс, минуя обработчики `atexit` и очистку буферов stdio. Применяется в детях при получении `SIGTERM`.

   (c) `getpid()` — возвращает PID текущего процесса (используется детьми для инициализации `srand`).

   (d) `getppid()` — возвращает PID родительского процесса; в детях применяется как адрес получателя сигналов `SIGUSR1`/`SIGUSR2`.

   (e) `waitpid(pid, *status, options)` — ожидает завершения конкретного потомка. В коде вызывается с `options = 0` для синхронного ожидания после рассылки `SIGTERM`.

2. **Работа с сигналами**

   (a) `signal(signum, handler)` — устанавливает обработчик сигнала. В родителе применяется для `SIGUSR1`, `SIGUSR2`, `SIGINT` и `SIGALRM`; в детях — для `SIGTERM`. Также используется `SIG_IGN` для игнорирования `SIGINT`, `SIGUSR1` и `SIGUSR2` в дочерних процессах.

   (b) `kill(pid, sig)` — посылает сигнал процессу с указанным PID. Дети шлют родителю `SIGUSR1`/`SIGUSR2`; родитель посылает детям `SIGTERM` при завершении.

   (c) `sigemptyset(set)` — инициализирует набор сигналов пустым множеством.

   (d) `sigaddset(set, signum)` — добавляет сигнал в набор. В коде добавляются `SIGUSR1` и `SIGUSR2` в маску блокировки.

   (e) `sigprocmask(how, set, oldset)` — изменяет маску блокируемых сигналов:

      i. `SIG_BLOCK` — добавляет сигналы в блокировку (начало закрытого окна);

      ii. `SIG_SETMASK` — восстанавливает прежнюю маску (конец окна).

   (f) `sigpending(set)` — заполняет набор сигналами, находящимися в состоянии ожидания доставки (в pending). В закрытом окне показывает наличие заблокированных сигналов.

   (g) `sigismember(set, signum)` — проверяет, входит ли сигнал в набор. Используется для вывода факта наличия `SIGUSR1`/`SIGUSR2` в pending (значение 0 или 1, а не количество).

(h) `sigsuspend(mask)` — атомарно заменяет маску сигналов процесса и приостанавливает его до получения любого сигнала, после чего восстанавливает исходную маску. Это единственный способ ожидания сигнала, исключающий гонку между проверкой условия и переходом в сон. В коде вызывается в открытом окне с пустой маской `empty`.

(i) `alarm(seconds)` — устанавливает таймер, по истечении которого процессу будет послан сигнал `SIGALRM`. Используется для гарантированного пробуждения родителя из `sigsuspend` раз в секунду, чтобы обновить проверку времени.

3. **Управление памятью и разделяемое состояние**

(a) `mmap(...)` — отображает разделяемую анонимную память (`MAP_SHARED | MAP_ANONYMOUS`) для счётчиков отправленных сигналов, доступных одновременно родителю и всем потомкам.

(b) `munmap(...)` — освобождает ранее отображённую область памяти перед завершением программы.

(c) `memset(...)` — обнуляет содержимое выделенной разделяемой памяти при инициализации.

4. **Время и задержки**

(a) `sleep(sec)` — приостанавливает процесс на указанное число секунд (закрытое окно).

(b) `usleep(usec)` — микросекундная задержка: интервал между сигналами у детей (случайный, 100–400 мс) и пауза после разблокировки для доставки сигналов.

(c) `alarm(1)` — устанавливает сигнал `SIGALRM` через одну секунду; применяется для периодического пробуждения родителя из `sigsuspend` и проверки длительности открытого окна.

(d) `time(NULL)` — возвращает текущее время в секундах; используется для контроля длительности открытого окна (`OPEN_SEC`).

5. **Генерация случайных чисел**

(a) `srand(seed)` — инициализирует генератор псевдослучайных чисел; в детях используется `getpid()`, чтобы у каждого процесса была своя последовательность.

(b) `rand()` — возвращает случайное число; применяется для выбора типа сигнала (`rand() & 1`) и случайной задержки ($100 +$ `rand()%300` мс).

6. **Ввод-вывод и стандартные функции**

(a) `setvbuf(stdout, NULL, _IONBF, 0)` — отключает буферизацию стандартного вывода, чтобы диагностические сообщения появлялись в консоли сразу.

(b) `printf(...)` — форматированный вывод статистики (начало и конец открытого окна, состояние pending, Δ обработанных сигналов, итоговая статистика).

7. **Пользовательские типы и макросы**

(a) `volatile sig_atomic_t` — тип, безопасно изменяемый в обработчиках сигналов. Используется для счётчиков `sent1`, `sent2` (в разделяемой памяти), а также `recv1`, `recv2`, `stop`.

(b) `pid_t` — тип идентификатора процесса.

(c) `sigset_t` — тип набора сигналов, применяемый в `sigemptyset`, `sigaddset`, `sigprocmask`, `sigpending`, `sigismember`, `sigsuspend`.

(d) `sigismember` возвращает ненулевое значение при наличии сигнала в наборе и нуль в противном случае.

8. **Общая схема работы**

(a) Родитель порождает $N = 3$ дочерних процесса через `fork()`. Дети наследуют код, но получают собственные PID и адресные пространства.

(b) Дети настраивают обработчики: `SIGINT`, `SIGUSR1`, `SIGUSR2` игнорируются, а `SIGTERM` завершает процесс через `_exit(0)`. Затем каждый ребёнок в бесконечном цикле со случайной задержкой 100–400 мс посылает родителю случайный сигнал (`SIGUSR1` или `SIGUSR2`), инкрементируя соответствующий счётчик в разделяемой памяти.

(c) Родитель выполняет 3 цикла, каждый из которых состоит из двух фаз:

i. *Открытое окно* (`OPEN_SEC` = 2 с): сигналы не заблокированы, обработчики срабатывают сразу. Ожидание реализовано через `sigsuspend(&empty)` с периодическим `alarm(1)`, что исключает гонку между проверкой условия и переходом в сон.

ii. *Закрытое окно* (`CLOSE_SEC` = 3 с): вызывается `sigprocmask(SIG_BLOCK, ...)`, и `SIGUSR1`/`SIGUSR2` копятся в pending. По окончании `sleep()` вызывается `sigpending()`, и с помощью `sigismember()` выводится факт наличия сигналов в pending. Затем маска восстанавливается через `SIG_SETMASK`, и доставляются отложенные сигналы.

(d) Сравнение `sent` (из разделяемой памяти) и `recv` (счётчик обработчиков) демонстрирует, что обычные сигналы не ставятся в очередь: за время блокировки в pending всегда находится только один бит на каждый тип, поэтому после разблокировки доставляется не более одного-двух экземпляров сигнала. Итоговые потери достигают 70–80%.

(e) По `SIGINT` родитель устанавливает флаг `stop = 1`, игнорирует дальнейшие `SIGINT`, рассылает детям `SIGTERM` и собирает их через `waitpid()`. Это обеспечивает корректное завершение без зомби-процессов и утечек ресурсов.

9. **Экспериментальные результаты**

(a) При `N` = 3, `OPEN` = 2 с, `CLOSE` = 3 с за три цикла было отправлено 99+78 = 177 сигналов, а обработано 19 + 19 = 38, то есть потеряно около 78%.

(b) В каждом закрытом окне `sigpending()` показывал `USR1=1`, `USR2=1` независимо от количества накопленных сигналов — это ключевое доказательство того, что pending хранит бит, а не счётчик.

(c) Приращение Δ после разблокировки всегда лежало в диапазоне 1–2: один экземпляр из pending плюс (возможно) один свежий сигнал, пришедший сразу после сброса бита.

(d) Эти данные полностью согласуются с моделью POSIX: для обычных сигналов (не real-time) повторные доставки одного типа сливаются в один pending-бит, тогда как сигналы реального времени (`SIGRTMIN`...`SIGRTMAX`), передаваемые через `sigqueue()`, ставятся в очередь и не теряются.